RBE104TC GROUP COURSEWORK

Warehouse Delivery Robot

Module	RBE104TC - C/C++ Programming Language
Assessment weighting	30% of the module mark
Assessment mode	Group coursework
Group size	3-4 students
Marks	100 marks, scaled to 30% of the module
Submission	Group report, source code and build/run guide
Report limit	Maximum 1,500 words
Deadline	To be determined

1. Project overview

Develop a terminal-based warehouse delivery game in standard C. The player controls a robot that collects packages and delivers them to a designated point while navigating walls and avoiding moving forklifts. The task is designed to assess practical programming, program design, testing and collaborative development.

2. Core requirements

2.1 Warehouse map and display

- Represent the warehouse using a two-dimensional character array.
- Use a clear set of symbols, for example: # for a wall, R for the robot, P for a package, D for the delivery point and F for a forklift.
- Display the current map and essential game information, including score, remaining lives and packages still to be delivered.
- A fixed map embedded in the program is sufficient. Additional map support is optional.

2.2 Controls and game loop

- Use W, A, S and D to move the robot and Q to quit.

- The robot must not move through walls or outside the map.
- After each valid player turn, every forklift makes one move.
- The game should continue until the player wins, loses all lives or quits.

2.3 Package collection and delivery

- The robot collects a package by moving onto its location.
- The robot may carry no more than one package at a time.
- A carried package is delivered when the robot reaches the delivery point, and the score is updated.
- The player wins when all packages have been delivered.

2.4 Forklifts and collisions

- Include between one and four forklifts.
- Each forklift selects a valid movement direction and must not pass through walls or leave the map.
- A collision between the robot and a forklift costs one life and returns the robot to its starting position.
- The player begins with three lives and loses the game when no lives remain.

2.5 Repeatability and input handling

- Allow a random seed to be supplied or entered so that a game can be repeated for testing.
- Handle invalid commands safely and provide clear feedback.
- The program should exit cleanly when the user quits or input ends.

3. Program design

You may choose the detailed implementation. The program should use suitable functions, arrays and data structures, with separate responsibilities for input, movement, collision checks, game-state updates and display. Clear names and a sensible file structure are expected. The submitted program must build and run in the stated module environment using a simple documented command.

4. Testing

Provide evidence of at least three repeatable test scenarios. Together, the tests should demonstrate successful package delivery, collision and life handling, and an important boundary, input, win or loss condition. Fixed seeds, concise console logs or equivalent evidence may be used. Explain what each test checks and whether the observed result is correct.

5. Optional extensions

After the core requirements work reliably, a group may add an appropriate extension, such as loading a map from a file, battery and charging rules, bonus packages, multiple levels, alternative forklift behaviour or a high-score feature. Complexity does not receive credit by itself, and an extension cannot compensate for missing or unreliable core functionality.

6. Group working

- Submit one integrated solution for the group.
- Include a concise contribution statement identifying each member's main responsibilities.
- All members are expected to understand the submitted program and participate in integration and testing.
- The program may be checked using different valid inputs, seeds or configurations.

7. Submission requirements

Item	Requirement
Group report	PDF, maximum 1,500 words.
Source code	All required .c and .h files; do not submit executable files as the only implementation.
Build/run guide	Short, accurate instructions including the required command and controls.
Testing evidence	Repeatable scenarios and concise results or console evidence.
Contribution statement	Main contribution of each group member.

8. Suggested report structure

- Overview of the solution and key design decisions.
- Program structure and representation of the game state.
- Implementation of the main game rules.
- Testing approach, evidence and discussion of results.
- Limitations and possible improvements.
- Group contribution statement.

9. Assessment and conduct

The work is marked using the accompanying 100-mark rubric and then scaled to the 30% module weighting. Marks reward a correct, understandable and well-tested solution rather than unnecessary complexity. Acknowledgement of external material or assistance should follow the guidance applicable to the module. Each group remains responsible for understanding, testing and accurately representing its submitted work.